TECHNICAL DECISION RECORD · VERSION 1.0

Tech *Stack*

Which framework, which database, and which free hosting Pothgula should be built on — and why the requirements themselves decide most of it.

PREPARED FOR	GOVERNS	DATE	STATUS
Chathuranga Jayawardhana	Requirements v2.0, §13	10 September 2026	Recommendation — for decision

This is not a requirements document. It proposes answers to two of the open questions in requirements v2.0 §18 — the open-source licence, and where the system will be hosted. Nothing here changes scope. Every hosting figure was verified on 10 September 2026; free tiers move, so section 7 records what to re-check and when.

CONTENTS

01 What has to be decided

02 What the requirements already decide

03 The licence audit

04 The recommended stack

05 How the pieces fit

06 Free hosting

07 Traps and things to re-check

08 What this closes in §18

09 Decisions needed

10 Sources

01

What has to be decided

Three choices, in the order that they constrain each other: the database, the API framework, and where the whole thing runs for free.

Pothgula is being built as a free and open-source project with a separate API and a separate front end. That separation is a deliberate design choice, not an accident of tooling — it means the library could later put a different front end, a kiosk screen or a phone app, on the same API without rebuilding the system.

What makes this decision unusually easy is that requirements v2.0 already contains three constraints strong enough to eliminate most of the field before any comparison begins. They are worth restating, because two of them are commonly overlooked and both are expensive to discover late.

> THE THREE GOVERNING REQUIREMENTS
>
> **NFR-11 — Cost.** No part of the system may depend on a paid third-party service, or on any charge per message, per request, per user or per month. This applies to reminders, password resets, maps, PDF generation, fonts, error reporting and analytics alike.
>
> **NFR-12 — Licence.** Every framework, database, library and tool used is free and open-source under a *permissive* licence, and the library owns and can modify its own copy without asking anyone.
>
> **NFR-13 — Hosting.** The whole system can be installed and run by the library on its own machine or on a low-cost server, with written setup instructions and *no vendor account required to run it*.

02

What the requirements already decide

NFR-13 means the architecture cannot be a platform

This is the constraint that most often gets missed, because the popular way to build a small system in 2026 is to lean on a platform that supplies authentication, storage and a database together. That is precisely what NFR-13 forbids. If the login flow is Supabase Auth, the system does not run without a Supabase account. If the API is a set of Vercel or Cloudflare functions, the system does not run without that vendor. In both cases the library cannot install its own copy, and the requirement fails.

The test to apply to every candidate is simple: **does the whole system come up with `docker compose up` on a laptop with no internet and no accounts anywhere?** If yes, the requirement holds. Free hosting then becomes just one deployment target among several, rather than the architecture itself.

RULED OUT BY NFR-13	WHY
Supabase as a platform	Auth, storage and realtime are the vendor's. Self-hosting Supabase is possible but is a far heavier operation than running a Postgres container.
Firebase	Not self-hostable at all, and not open source.
Cloudflare D1 or a Workers-only back end	The runtime and the database exist only inside Cloudflare.
A serverless-shaped API	Ties request handling to one vendor's runtime and deployment model.
Auth0, Clerk, Firebase Auth	Vendor account required, and every one of them charges above a user threshold — so NFR-11 as well.

Note that using Supabase or Neon *as a plain hosted Postgres* is entirely fine, because the connection string can be pointed at a local container instead. What fails the test is depending on a vendor's *features*, not on their hosting.

NFR-11 reaches further than it first appears

The requirement names fonts and PDF generation explicitly, which has three practical consequences that need deciding now rather than during construction:

- **Fonts.** Sinhala and Tamil rendering under NFR-04 needs Noto Sans Sinhala and Noto Sans Tamil. These are free under the SIL Open Font License, but they must be *committed to the repository and served by the application*, not pulled from the Google Fonts CDN — otherwise a self-hosted install without internet renders member names as empty boxes.
- **PDF and reports.** Section 12 requires printable summaries. Generate them in the browser with print stylesheets, or client-side with a permissively licensed library. No PDF-as-a-service.
- **Error reporting.** No Sentry or equivalent. Errors go to the application's own log, which the library's own server keeps.

NFR-10 decides the database category

The requirement is that the same book cannot be issued twice at the same moment even if two staff act at once, and that two branches cannot be given the same accession number — *enforced in the database, not only in the screen*. That is a direct instruction to use a relational database with real transactions and real constraints, and to express both rules as constraints rather than as application code:

RULE	HOW IT IS ENFORCED
An accession number is unique across the whole organisation	A unique index on the accession number column. The `AA00001` format already carries the branch letter, so collisions are structurally unlikely — but the index is what makes it certain.
A copy cannot be on two open loans at once	A **partial unique index** on the copy's identifier, restricted to rows where the loan is still open. Two simultaneous issues then become one success and one database error, with no application locking to get wrong.

RULE	HOW IT IS ENFORCED
A member may hold at most two books, per v2.0 Q5	Checked inside the same transaction as the issue, with the member's rows locked for the duration.

03

The licence audit

NFR-12 asks for permissive licences specifically, not merely open source. That distinction disqualifies several obvious defaults.

A permissive licence — MIT, BSD, Apache 2.0, the PostgreSQL licence, the SIL Open Font License — lets the library use, modify and redistribute its copy with essentially no obligation beyond keeping the notice. A copyleft licence such as GPL or AGPL attaches conditions to redistribution; SSPL is not recognised as open source at all. For a system a library may one day hand to another library, the difference matters.

COMPONENT	LICENCE	NFR-12	NOTE
PostgreSQL	PostgreSQL licence	**Pass**	Permissive, BSD-like. The recommended database.
MySQL	GPLv2	**Fail**	Copyleft. Also Oracle-owned, with a paid edition alongside.
MariaDB	GPLv2	**Fail**	Copyleft, despite being the community fork.
MongoDB	SSPL	**Fail**	Not OSI-approved. Not open source by the usual definition.
Redis, version 8	AGPLv3 / RSALv2	**Fail**	Relicensed away from BSD in 2024. Use **Valkey**, BSD-3, if a cache is ever needed — at fifty concurrent users, it is not.
SQLite	Public domain	**Pass**	Passes on licence, but see the note below.
Node.js	MIT	**Pass**	
NestJS	MIT	**Pass**	
React, Vite, Tailwind CSS	MIT	**Pass**	
Drizzle ORM	MIT	**Pass**	
Noto Sans Sinhala and Tamil	SIL OFL 1.1	**Pass**	Permissive for embedding. Commit the files; do not use the CDN.
Docker Engine	Apache 2.0	**Pass**	Docker Desktop's licence differs, but the engine is what a server runs.

> **WHY NOT SQLITE, GIVEN IT PASSES**
>
> SQLite is licence-clean and would simplify hosting considerably. It is not recommended here because NFR-10 asks for correctness under simultaneous writes from multiple staff, and SQLite serialises writers across the whole database. At fifty concurrent users it would very likely cope — but Postgres removes the question entirely, costs nothing, and is what the library would want on the day a fourth branch opens.

04

The recommended stack

TypeScript on both sides of the API boundary, PostgreSQL underneath, everything shipped as containers.

LAYER	CHOICE	LICENCE	WHY THIS ONE
Database	PostgreSQL 17	PostgreSQL	The only mainstream relational database with a permissive licence. Gives NFR-10 its partial unique indexes, NFR-04 its UTF-8 handling of Sinhala and Tamil, and NFR-14 its exact integer money as `bigint` cents.
API	Node 22 with NestJS	MIT	Its module, guard and dependency-injection structure maps directly onto the three roles and onto branch scoping: a single guard enforces NFR-06's rule that the branch a staff member may act on is checked on the server for every request. Opinionated structure is a virtue on an open-source project, where a newcomer has to find their way around unaided.
Database access	Drizzle ORM	MIT	SQL-first, so the NFR-10 constraints are written explicitly in migrations rather than left to an ORM's discretion. Lighter in memory than the alternatives, which matters on a 512 MB free instance.
Validation	Zod	MIT	One schema per request and response, shared by both sides — see section 5. This is the main practical payoff of choosing one language for a split API and front end.
Authentication	Self-issued JWT, argon2id hashing	MIT	NFR-06 requires modern password hashing; argon2id is the current recommendation. Tokens are issued by the application itself, so there is no identity vendor and no NFR-13 violation. Password reset happens at the counter, per v2.0 §4, so no email or SMS provider is needed anywhere in the system.
Front end	React with Vite, TypeScript	MIT	Builds to plain static files that any web server can serve. Deliberately *not* Next.js: server-side rendering would add a second runtime to install and keep alive, which works against NFR-13 for no benefit an internal library system needs.
Interface	Tailwind CSS with shadcn/ui	MIT	shadcn/ui is copied into the repository rather than installed as a dependency, so the library genuinely owns its interface code — which is what NFR-12's second clause asks for.
Languages	react-i18next	MIT	Sinhala, Tamil and English under NFR-04, with the Noto fonts served from the application itself.

LAYER	CHOICE	LICENCE	WHY THIS ONE
Packaging	Docker Compose	Apache 2.0	Three containers — API, static front end, Postgres — and one command. This *is* the NFR-13 deliverable.
Build and release	GitHub Actions and GHCR	—	Free and unlimited for public repositories, including public container images. The image the library self-hosts is then the same image that was tested.

Why not .NET, Java or Python

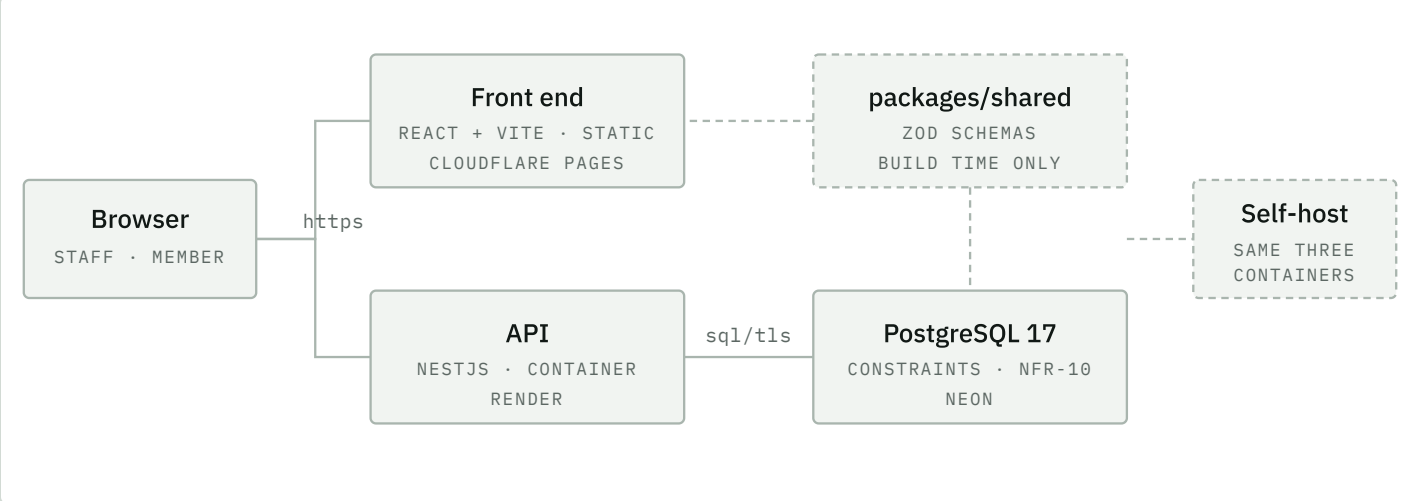
All three are licence-clean and all three would produce a good system. The recommendation is driven by the free hosting tier, not by quality: a free container gets 512 MB of memory and a tenth of a processor, and it is stopped and restarted whenever traffic pauses. A Node process starts in that environment in about a second; a JVM or .NET process is slower to start and heavier at rest, which turns every first visit to the demonstration into a noticeably long wait.

If Pothgula is eventually hosted on a real server — which section 6 recommends anyway — that argument disappears entirely, and any of the three would be a sound choice. The other consideration is smaller but real: with TypeScript on both sides, the API's request and response types are the front end's types, checked by the compiler. That is worth a good deal on a split project maintained by one person.

05

How the pieces fit

One repository, three deployable pieces, and a shared package of schemas in the middle that keeps the API and the front end honest with each other.



The solid path is the free hosted deployment. The dashed box on the right is the same three pieces running from one `docker compose` file on the library's own machine — the NFR-13 deliverable, and the one that matters at handover.

Repository layout

PATH	WHAT LIVES THERE
`apps/api`	NestJS application, Drizzle migrations, seed data. Ships as a container image.
`apps/web`	React and Vite front end. Builds to static files.
`packages/shared`	Zod schemas and the types derived from them. Imported by both sides, so an API change the front end has not caught up with fails the build rather than production.
`infra/`	`docker-compose.yml`, the backup and restore scripts NFR-08 requires, and the written setup instructions NFR-13 requires.
`docs/`	The requirements documents and this one.

A single repository is recommended over separate API and front-end repositories, despite the deployments being separate. The shared schema package is the reason: across two repositories it would have to be published to a registry and version-bumped on every change, which is a great deal of ceremony for a project with one maintainer.

06

Free hosting

Three services, none requiring a credit card, none expiring. Verified 10 September 2026.

PIECE	SERVICE	WHAT THE FREE TIER GIVES	THE CATCH
Front end	Cloudflare Pages	Unmetered requests for static assets, 500 builds a month, free custom domain and TLS.	None that applies here. The 100,000-requests-a-day ceiling belongs to Workers functions, which a purely static build never touches.
API	Render	750 instance-hours a month, 512 MB memory, no credit card.	Spins down after 15 minutes idle; the next request then waits roughly a minute while it starts. See R-02.
Database	Neon	0.5 GB storage, scales to zero and resumes instantly, no expiry, no credit card.	Storage is per project, and exceeding the free allowance suspends compute until the next month rather than deleting anything.

> **DO NOT USE RENDER'S FREE POSTGRES**
>
> Render offers a free PostgreSQL database alongside its free web service, and it is the obvious thing to reach for. It **expires 30 days after creation**, with a 14-day grace period, after which the database and its data are deleted. That is fine for a week's experiment and wrong for anything that must still be there next term. Render for the API, Neon for the database.

Is half a gigabyte enough

Comfortably. The accession number format allows 26 branches, and a branch library holds a few thousand copies. Even at the ceiling — 26 branches, five thousand copies each, twenty thousand members, and a decade of loan history and activity log — the whole database sits well under a tenth of the allowance. Storage will not be the constraint that forces a paid plan, and at fifty concurrent users nothing else in the free tier will be either.

The alternative worth building towards

A single always-free virtual machine on Oracle Cloud, running the same `docker compose` file, replaces all three services with one. It does not spin down, so the two-second target in NFR-01 holds for every request, and it doubles as a live proof that the self-hosted path in NFR-13 actually works — which is the thing most likely to be quietly broken at handover if nobody ever exercises it.

Three caveats. Oracle halved the always-free ARM allocation in June 2026, from four processors and 24 GB of memory to two and 12 GB — still far more than this system needs, but an existing instance must be resized to the new shape or it is stopped. Signup requires a card for identity verification, though the always-free resources themselves are not charged. And idle instances can be reclaimed, so a lightly loaded server needs something periodic keeping it awake.

The suggested order is to start on the three free services, because nothing there needs a card or an approval and the demonstration can be live the same day, then move to the virtual machine once the compose file exists and is worth exercising.

Two options the guides still recommend that should not be used

- **Koyeb** closed its free tier to new signups after the Mistral acquisition in February 2026.
  Existing accounts keep it; new ones cannot be created.
- **Fly.io** no longer offers a free tier to new users, and requires a card.

Both still appear near the top of most "best free hosting" articles, which is worth remembering when re-checking these figures later: the comparison sites lag the platforms by months.

Traps and things to re-check

Safari blocks the obvious login mechanism R-01

A separate API and front end means two different origins. The usual session cookie then has to be marked `SameSite=None`, which Safari treats as a third-party cookie and blocks — and NFR-03 requires Safari support. Members will mostly be on phones, where Safari is not a minority.

Decide before writing the authentication module. Either send the token in an `Authorization` header, which works everywhere and is the simpler choice, or put both pieces behind one domain with a proxy route so the cookie is first-party. Retrofitting either onto the other means revisiting every request path.

Cold starts contradict NFR-01 R-02

NFR-01 requires any counter action to complete in under two seconds. A free Render instance that has been idle fifteen minutes takes about a minute to answer its first request. Read literally, the free tier cannot meet the requirement.

Write the distinction down now. The free hosting serves the public demonstration and the open-source showcase; the library's counter runs self-hosted, where NFR-01 is met with room to spare. Recorded before acceptance this is a deployment note; discovered at acceptance it reads as a missed requirement.

Free tiers move faster than documents R-03

Within the last year Fly.io withdrew its free tier, Koyeb closed signups, and Oracle halved its allocation without announcing it. Every figure in section 6 is dated 10 September 2026 for that reason.

Re-check before each phase begins, and keep the compose file working throughout. Because every piece is self-hostable, a platform withdrawing its free tier costs a redeployment rather than a rewrite — which is the practical reason NFR-13 is worth honouring strictly.

Backups are a requirement, not an afterthought R-04

NFR-08 requires a daily backup, a restore tested before handover, and free self-hosted tooling. Neon's free plan keeps a six-hour history window, which is not the same thing as a backup you hold yourself.

Put `pg_dump` on a schedule, in the repository, from the first week, writing somewhere the library controls. It is a few lines, it satisfies NFR-08 on both the hosted and self-hosted deployments, and the restore rehearsal is the part that gets skipped when it is left until the end.

Sinhala and Tamil are a build concern R-05

NFR-04 requires all three languages in every field, including names and book titles. Postgres handles the storage without special configuration. Rendering is where it breaks — and it breaks on the machine that has no internet, which is the self-hosted one.

Commit the Noto font files and serve them from the application. Put real Sinhala and Tamil names in the seed data from the first sprint, so the problem appears in development rather than at the counter.

Personal data would sit on someone else's server R-06

NFR-09 places member names, phone numbers, dates of birth and addresses under Sri Lanka's Personal Data Protection Act No. 9 of 2022. The free hosted deployment would put that data on infrastructure outside the country.

Keep real member data off the free deployment entirely. Seed the public demonstration with invented records. Real data belongs only on the library's own self-hosted install — which is a further argument for exercising that path early.

08

What this closes in §18

Two of the open questions in requirements v2.0 have answers if this recommendation is accepted.

QUESTION	PROPOSED ANSWER
Which open-source licence should the project carry, and who holds the copyright — the library, or you personally? `NFR-12`	**MIT.** It is consistent to offer outward the same permissiveness NFR-12 demands of every dependency, and it is the licence most likely to let another library adopt the system without needing to take advice first. Who holds the copyright is a separate question that only you can answer; it does not affect the technical choice.
Where will the system be hosted, and who will run the daily backup and test the restore? `NFR-08 · NFR-13`	**Two deployments.** A public demonstration on Cloudflare Pages, Render and Neon, seeded with invented data. The library's real install self-hosted from the compose file, with `pg_dump` on a daily schedule. Who runs the backup remains a question for the client — it is an operational commitment rather than a technical one, and R-04 explains why the restore rehearsal in particular must be scheduled.

09

Decisions needed

Five, in the order they block work.

Q1 **Accept the stack?**

PostgreSQL, NestJS, React with Vite, everything in containers. Nothing else can start until this is settled.

Q2 **Header tokens or a proxied cookie for authentication?**

R-01. This shapes every request path in the system and is expensive to change later. The recommendation is header tokens.

Q3 **Is MIT the licence, and in whose name?**

The licence file belongs in the first commit; adding it later means asking every contributor to agree.

Q4 **Is the distinction between demonstration hosting and the library's own install accepted?**

R-02. It needs to be written into the requirements before acceptance rather than explained at it.

Q5 **Who runs the backup and the restore rehearsal?**

NFR-08 names the requirement

but
not
the
person.
This
is a
client
question.

Questions one, two and three block the first line of code. Four and five can be settled during phase one, but not later than it.

10

Sources

Hosting figures verified 10 September 2026. Licences verified against each project's own repository.

Render — Deploy for Free · render.com/docs/free

Render free tier and the 30-day database · justinmckelvey.com/blog/is-render-free

Free PostgreSQL hosting, maintained comparison · github.com/freebase-cloud/free-postgres-hosting

Supabase and Neon free tiers compared · toolfreebie.com/supabase-vs-neon

Cloudflare Pages free tier limits, 2026 · temps.sh/blog/cloudflare-pages-free-tier-limits-2026

Koyeb free tier status after the acquisition · srvrlss.io/provider/koyeb

Free Docker hosting platforms tested, March 2026 · snapdeploy.dev/blog/free-docker-hosting-2026-platforms-compared

Oracle halves always-free Ampere A1 limits · infoq.com/news/2026/07/oracle-cloud-free-tier-limits

Oracle Cloud Infrastructure Free Tier documentation · docs.oracle.com/iaas/Content/FreeTier/freetier.htm

---